CHỦ ĐỀ DELEGATE & EVENT

1. Bài toán nghiệp vụ

Trường đại học quản lý các thiết bị trong phòng dạy:

- Điều khiển máy chiếu.
- Điều khiển điều hòa.
- Micro không dây.

Thiết bị có thể phát sinh các thay đổi cần được các thành phần khác biết đến:

- Mức pin xuống thấp.
- Trạng thái chuyển sang Borrowed, Available hoặc UnderMaintenance.
- Micro có chất lượng tín hiệu thấp.

Hệ thống có nhiều thành phần quan tâm đến các sự kiện này:

- ConsoleNotificationService: hiển thị cảnh báo.
- MaintenanceService: tạo yêu cầu bảo trì khi có lỗi nghiêm trọng.
- DeviceAuditLogger: ghi lại lịch sử thay đổi.
- Trong tương lai có thể bổ sung email, ứng dụng di động hoặc dashboard mà không sửa lớp thiết bị.

Bài toán thiết kế trọng tâm:

Thiết bị chỉ chịu trách nhiệm phát hiện và công bố điều đã xảy ra. Thiết bị không nên biết cụ thể ai sẽ ghi log, gửi cảnh báo hoặc tạo yêu cầu bảo trì. => Áp dụng delegate và event.

2. Thiết kế trước

Không bắt đầu cài đặt toàn bộ trước khi có thiết kế. Một số gợi ý thiết kế:

1. Xác định publisher và subscriber.
2. Xác định các event cần thiết.
3. Xác định dữ liệu đi kèm mỗi event.
4. Vẽ sơ đồ lớp UML.
5. Trả lời câu hỏi:
    - Vì sao thiết bị không nên gọi trực tiếp MaintenanceService?
    - Khi nào cần một event riêng, khi nào có thể dùng chung một event?
    - Thành phần nào có quyền phát sinh event?

3. Danh mục

Trạng thái thiết bị - enum

```
public enum DeviceStatus
{
    Available,
    Borrowed,
    UnderMaintenance
}
```

Mức độ cảnh báo - enum

```
public enum AlertSeverity
{
    Information,
    Warning,
    Critical
}
```

4. Class/lớp dữ liệu sự kiện

Sự kiện thay đổi trạng thái

```
public class DeviceStatusChangedEventArgs : EventArgs
{
    public string DeviceId { get; }
    public DeviceStatus OldStatus { get; }
    public DeviceStatus NewStatus { get; }
    public DateTime ChangedAt { get; }

    public DeviceStatusChangedEventArgs(
        string deviceId,
        DeviceStatus oldStatus,
        DeviceStatus newStatus,
        DateTime changedAt)
    {
        DeviceId = deviceId;
        OldStatus = oldStatus;
        NewStatus = newStatus;
        ChangedAt = changedAt;
    }
}
```

Sự kiện cảnh báo thiết bị

```
public class DeviceAlertEventArgs : EventArgs
{
    public string DeviceId { get; }
    public string Message { get; }
    public AlertSeverity Severity { get; }
    public DateTime OccurredAt { get; }
```

```
    public DeviceAlertEventArgs(
        string deviceId,
        string message,
        AlertSeverity severity,
        DateTime occurredAt)
    {
        DeviceId = deviceId;
        Message = message;
        Severity = severity;
        OccurredAt = occurredAt;
    }
}
```

5. Class/lớp trừu tượng TeachingDevice

```
public abstract class TeachingDevice
{
    public string Id { get; }
    public string Name { get; }
    public DeviceStatus Status { get; private set; }

    public event EventHandler<DeviceStatusChangedEventArgs>?
        StatusChanged;

    public event EventHandler<DeviceAlertEventArgs>?
        AlertRaised;

    protected TeachingDevice(string id, string name)
    {
        Id = id;
        Name = name;
        Status = DeviceStatus.Available;
    }

    public void ChangeStatus(DeviceStatus newStatus);

    protected void RaiseAlert(
        string message,
        AlertSeverity severity);

    protected virtual void OnStatusChanged(
        DeviceStatusChangedEventArgs e);

    protected virtual void OnAlertRaised(
        DeviceAlertEventArgs e);
}
```

Quy tắc thiết kế

- Event chỉ được phát sinh từ bên trong TeachingDevice hoặc lớp dẫn xuất.
- Mã bên ngoài được đăng ký và hủy đăng ký nhưng không được tự phát sinh event.
- ChangeStatus() không phát sinh event nếu trạng thái mới bằng trạng thái cũ.

- Phải cập nhật trạng thái rồi mới phát sinh StatusChanged.
- Các phương thức On... là protected virtual để lớp dẫn xuất có thể mở rộng hợp lý.
- TeachingDevice không tham chiếu trực tiếp đến logger, dịch vụ bảo trì hoặc giao diện người dùng.

Cài đặt khung

```
public void ChangeStatus(DeviceStatus newStatus)
{
    if (newStatus == Status)
    {
        return;
    }

    DeviceStatus oldStatus = Status;
    Status = newStatus;

    OnStatusChanged(
        new DeviceStatusChangedEventArgs(
            Id,
            oldStatus,
            newStatus,
            DateTime.Now));
}
protected virtual void OnStatusChanged(
    DeviceStatusChangedEventArgs e)
{
    StatusChanged?.Invoke(this, e);
}
protected virtual void OnAlertRaised(
    DeviceAlertEventArgs e)
{
    AlertRaised?.Invoke(this, e);
}
```

6. Class/ lớp thiết bị

ProjectorRemote

```
public class ProjectorRemote : TeachingDevice
{
    public int BatteryLevel { get; private set; }

    public void UpdateBatteryLevel(int newLevel);
}
```

Quy tắc xử lý:

- Mức pin phải từ 0 đến 100.
- Khi pin giảm xuống dưới 20%, phát sinh cảnh báo Warning.

- Khi pin bằng 0%, phát sinh cảnh báo Critical.
- Không phát sinh cảnh báo nếu cập nhật pin nhưng vẫn ở cùng một vùng an toàn, trừ khi sinh viên có lập luận khác.

AirConditionerRemote

```
public class AirConditionerRemote : TeachingDevice
{
    public string AssignedRoom { get; }
    public bool IsResponsive { get; private set; }

    public void UpdateResponseState(bool isResponsive);
}
```

Quy tắc xử lý:

- Khi chuyển từ phản hồi sang không phản hồi, phát sinh cảnh báo Critical.
- Không phát sinh lặp lại nếu trạng thái vẫn là không phản hồi.

WirelessMicrophone

```
public class WirelessMicrophone : TeachingDevice
{
    public int BatteryLevel { get; private set; }
    public int SignalQuality { get; private set; }

    public void UpdateBatteryLevel(int newLevel);
    public void UpdateSignalQuality(int newQuality);
}
```

Quy tắc xử lý:

- Pin dưới 30% phát sinh cảnh báo Warning.
- Chất lượng tín hiệu dưới 40% phát sinh cảnh báo Warning.
- Pin bằng 0 hoặc tín hiệu bằng 0 phát sinh cảnh báo Critical.

7. Các subscriber

ConsoleNotificationService

```
public class ConsoleNotificationService
{
    public void HandleAlert(
        object? sender,
        DeviceAlertEventArgs e)
    {
        Console.WriteLine(
            $"[{e.Severity}] {e.DeviceId}: {e.Message}");
```

```
    }
}
```

DeviceAuditLogger

```
public class DeviceAuditLogger
{
    private readonly List<string> entries = new();

    public IReadOnlyList<string> Entries => entries.AsReadOnly();

    public void HandleStatusChanged(
        object? sender,
        DeviceStatusChangedEventArgs e)
    {
        entries.Add(
            $"{e.ChangedAt}: {e.DeviceId}, " +
            $"{e.OldStatus} -> {e.NewStatus}");
    }
}
```

MaintenanceService

```
public class MaintenanceService
{
    private readonly List<string> requests = new();

    public IReadOnlyList<string> Requests => requests.AsReadOnly();

    public void HandleCriticalAlert(
        object? sender,
        DeviceAlertEventArgs e)
    {
        if (e.Severity != AlertSeverity.Critical)
        {
            return;
        }

        requests.Add(
            $"Maintenance request for {e.DeviceId}: {e.Message}");
    }
}
```

Yêu cầu

- Một event có thể có nhiều subscriber.
- Một subscriber có thể chỉ quan tâm một phần dữ liệu sự kiện.
- Publisher không thay đổi khi bổ sung subscriber mới.
- Subscriber không truy cập trực tiếp trường nội bộ của publisher.

8. Delegate và callback cho chính sách lọc thông báo

Ngoài event, hệ thống sử dụng delegate để minh họa callback có chủ đích.

Delegate tự định nghĩa

```
public delegate bool AlertFilter(DeviceAlertEventArgs alert);
```

Xây dựng lớp:

```
public class FilteredAlertSubscriber
{
    private readonly AlertFilter filter;
    private readonly Action<DeviceAlertEventArgs> action;

    public FilteredAlertSubscriber(
        AlertFilter filter,
        Action<DeviceAlertEventArgs> action)
    {
        this.filter = filter;
        this.action = action;
    }

    public void HandleAlert(
        object? sender,
        DeviceAlertEventArgs e)
    {
        if (filter(e))
        {
            action(e);
        }
    }
}
```

Ví dụ sử dụng

```
var criticalOnly = new FilteredAlertSubscriber(
    alert => alert.Severity == AlertSeverity.Critical,
    alert => Console.WriteLine(
        $"CRITICAL: {alert.DeviceId} - {alert.Message}"));
```

9. Class/lớp điều phối đăng ký

Để tránh rải logic += và -= ở nhiều nơi, có thể thiết kế:

```
public class DeviceMonitoringCoordinator : IDisposable
{
    private readonly List<TeachingDevice> devices;
    private readonly ConsoleNotificationService notificationService;
    private readonly DeviceAuditLogger auditLogger;
    private readonly MaintenanceService maintenanceService;

    public DeviceMonitoringCoordinator(
        IEnumerable<TeachingDevice> devices,
        ConsoleNotificationService notificationService,
        DeviceAuditLogger auditLogger,
        MaintenanceService maintenanceService);

    public void Subscribe();
    public void Unsubscribe();
    public void Dispose();
}
```

Yêu cầu

- Subscribe() đăng ký các handler cần thiết.
- Không đăng ký trùng cùng handler nhiều lần.
- Unsubscribe() hủy đúng các đăng ký đã tạo.
- Dispose() gọi Unsubscribe().

10. Ngoại lệ và tính ổn định của event

Xây dựng tối thiểu:

```
public class InvalidDeviceReadingException : Exception
{
    public string DeviceId { get; }
    public int InvalidValue { get; }
}
```

Phát sinh khi pin hoặc tín hiệu nằm ngoài khoảng 0–100.